Day 4 - Advanced Python Concepts

Prepared by: Sarthak Jejurkar

1. Python Concepts

Topic	Simple meaning	Example
List	Ordered, changeable collection.	`[88, 93, 55]`
Tuple	Ordered collection that cannot be changed.	`(10, 20)`
Set	Unordered collection of unique items.	`{1, 2, 3}`
Dictionary	Stores data as key-value pairs.	`{"name": "Aarav"}`
String	Text with useful methods.	`text.strip().title()`
Function / module	Reusable block of code / ready-made code file.	`def average(); import math`
Exception handling	Handles errors without crashing the program.	`try / except`
File handling	Reads and writes data in files.	`open(file, "w")`
Comprehension	Builds a list or dict in one line.	`[n*n for n in nums]`
Lambda / map / filter	Small one-line functions for processing data.	`lambda m: m >= 60`

2. Program 1: Collection-Based Practice

Practises lists, tuples, sets, dictionaries, strings, functions, comprehensions and lambda.

```
import math                                  # module

# Lists and tuples
marks = [88, 93, 55, 71]
marks.append(80)
point = (10, 20)                             # tuple (cannot be changed)
print("List:", marks, "| Tuple:", point)

# Sets and dictionaries
unique = set([1, 2, 2, 3, 3])                # removes duplicates
student = {"id": 1, "name": "Aarav", "marks": 88}
print("Set:", unique, "| Dict name:", student["name"])

# Strings
text = "  python programming  "
print(text.strip().title(), "| Length:", len(text.strip()))


# Functions and exception handling
```

```
def average(nums):
    return sum(nums) / len(nums)


try:
    print("Average:", average(marks))
    print(average([]))
except ZeroDivisionError:
    print("Error: list is empty")

# List and dictionary comprehensions
squares = [n * n for n in range(1, 6)]
result = {m: ("Pass" if m >= 40 else "Fail") for m in marks}
print("Squares:", squares)
print("Result:", result)

# Lambda, map, filter
passed = list(filter(lambda m: m >= 60, marks))
doubled = list(map(lambda m: m * 2, marks))
print("Passed 60+:", passed)
print("Doubled:", doubled)
print("Sorted:", sorted(marks, reverse=True), "| sqrt(16):", math.sqrt(16))
```

Output

```
List: [88, 93, 55, 71, 80] | Tuple: (10, 20)
Set: {1, 2, 3} | Dict name: Aarav
Python Programming | Length: 18
Average: 77.4
Error: list is empty
Squares: [1, 4, 9, 16, 25]
Result: {88: 'Pass', 93: 'Pass', 55: 'Pass', 71: 'Pass', 80: 'Pass'}
Passed 60+: [88, 93, 71, 80]
Doubled: [176, 186, 110, 142, 160]
Sorted: [93, 88, 80, 71, 55] | sqrt(16): 4.0
```

3. Program 2: Student Data Processing App

Stores student records in a list of dictionaries, saves them to `students.txt`, and lets you search by ID or name and sort by name or marks. Invalid input is handled with `try / except`.

```
FILE = "students.txt"


def load():
    students = []
    try:
        with open(FILE) as f:                  # read from file
            for line in f:
                sid, name, marks = line.strip().split(",")
                students.append({"id": int(sid), "name": name, "marks": int(marks)})
    except FileNotFoundError:
        pass
    return students


def save(students):
```

```
    with open(FILE, "w") as f:                # write to file
        for s in students:
            f.write(f"{s['id']},{s['name']},{s['marks']}\n")


def show(students):
    if not students:
        print("No records found.")
        return
    print("ID   Name        Marks")
    for s in students:
        print(f"{s['id']:<5}{s['name']:<12}{s['marks']}")


students = load()
while True:
    print("\n1.Add  2.View  3.Search  4.Sort  5.Exit")
    choice = input("Enter choice: ")
    try:
        if choice == "1":
            sid = int(input("ID: "))
            name = input("Name: ")
            marks = int(input("Marks: "))
            if not 0 <= marks <= 100:
                raise ValueError
            students.append({"id": sid, "name": name, "marks": marks})
            save(students)
            print("Record saved.")
        elif choice == "2":
            show(students)
        elif choice == "3":
            key = input("Enter ID or name: ").lower()
            show([s for s in students
                  if str(s["id"]) == key or s["name"].lower() == key])
        elif choice == "4":
            by = input("Sort by (name/marks): ")
            if by == "name":
                show(sorted(students, key=lambda s: s["name"]))
            elif by == "marks":
                show(sorted(students, key=lambda s: s["marks"], reverse=True))
            else:
                print("Invalid option.")
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            print("Invalid choice.")
    except ValueError:
        print("Invalid input! Enter valid numbers (marks 0-100).")
```

Sample run

```
1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 1
ID: 1
Name: Aarav
Marks: 88
Record saved.

1.Add  2.View  3.Search  4.Sort  5.Exit
```

```
Enter choice: 1
ID: 2
Name: Diya
Marks: 93
Record saved.

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 1
ID: 3
Name: Kabir
Marks: 55
Record saved.

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 1
ID: 4
Name: Meera
Marks: abc
Invalid input! Enter valid numbers (marks 0-100).

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 2
ID   Name        Marks
1    Aarav       88
2    Diya        93
3    Kabir       55

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 3
Enter ID or name: diya
ID   Name        Marks
2    Diya        93

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 4
Sort by (name/marks): marks
ID   Name        Marks
2    Diya        93
1    Aarav       88
3    Kabir       55

1.Add  2.View  3.Search  4.Sort  5.Exit
Enter choice: 5
Goodbye!
```